

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO2:** Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr.T.P.Anithaashri

QUESTIONS: 10

Total Marks: 50

Annexure A **Application - Based Questions**

Duration: 2Hrs

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality. **(5 Marks)**
2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations. **(5 Marks)**
3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time. **(5 Marks)**
4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance. **(5 Marks)**
5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism. **(5 Marks)**

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes. **(5 Marks)**

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization. **(5 Marks)**

8. **Design and develop** a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency. **(5 Marks)**

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance. **(5 Marks)**

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function. **(5 Marks)**

Code of Conduct:

*I K.Vijay Bhaskar Reddy(192425155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q1. Design and develop a Reinforcement Learning framework for a ride-sharing company that uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality. (5 Marks)

Aim

To design a Reinforcement Learning framework for efficient driver-passenger matching.

Theory

Reinforcement Learning helps a ride-sharing system assign the best driver to a passenger based on location, traffic, and availability. The agent learns from rewards to reduce waiting time and improve customer satisfaction.

MDP Components

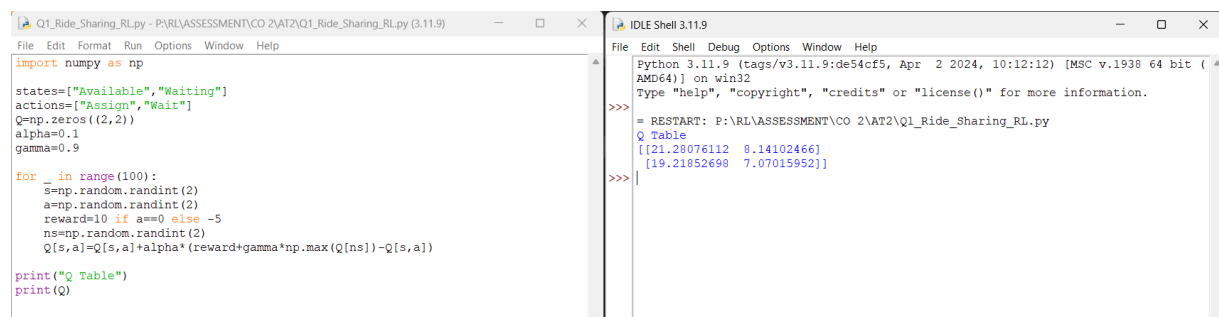
States: Driver Available, Passenger Waiting

Actions: Assign Driver, Wait

Rewards: +10 for quick pickup, -5 for long waiting time

Policy: Choose the driver with the highest expected reward.

Code & Output



The screenshot shows a Python script in a file editor and its execution output in a terminal window. The script defines a simple Reinforcement Learning environment for driver-passenger matching. It initializes a Q-table and simulates 100 episodes, updating the Q-values based on rewards. The output displays the final Q-table values.

```
Q1_Ride_Sharing_RL.py - P:\RL\ASSESSMENT\CO 2\AT2\Q1_Ride_Sharing_RL.py (3.11.9)
File Edit Format Run Options Window Help
import numpy as np

states=["Available","Waiting"]
actions=["Assign","Wait"]
Q=np.zeros((2,2))
alpha=0.1
gamma=0.9

for _ in range(100):
    s=np.random.randint(2)
    a=np.random.randint(2)
    reward=10 if a==0 else -5
    ns=np.random.randint(2)
    Q[s,a]=Q[s,a]+alpha*(reward+gamma*np.max(Q[ns])-Q[s,a])

print("Q Table")
print(Q)
```

```
IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q1_Ride_Sharing_RL.py
Q Table
[[21.28076112  8.14102466]
 [19.21852698  7.07015952]]
>>>
```

Code & Output

1. Define driver and passenger states.
2. Define available actions.
3. Initialize the Q-table.
4. Train using rewards.
5. Update Q-values.
6. Display the learned policy.

Advantages

- Reduces waiting time.
- Improves driver utilization.
- Learns automatically.
- Enhances customer satisfaction.

Applications

- Uber
- Ola
- Rapido
- Taxi booking services

Conclusion

RL improves ride-sharing efficiency by learning optimal driver assignment strategies.

Q2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations. (5 Marks)

Aim

To understand exploration and exploitation in personalized learning.

Theory

RL recommends study materials by balancing familiar topics (exploitation) and new topics (exploration). This improves learning effectiveness over time.

MDP Components

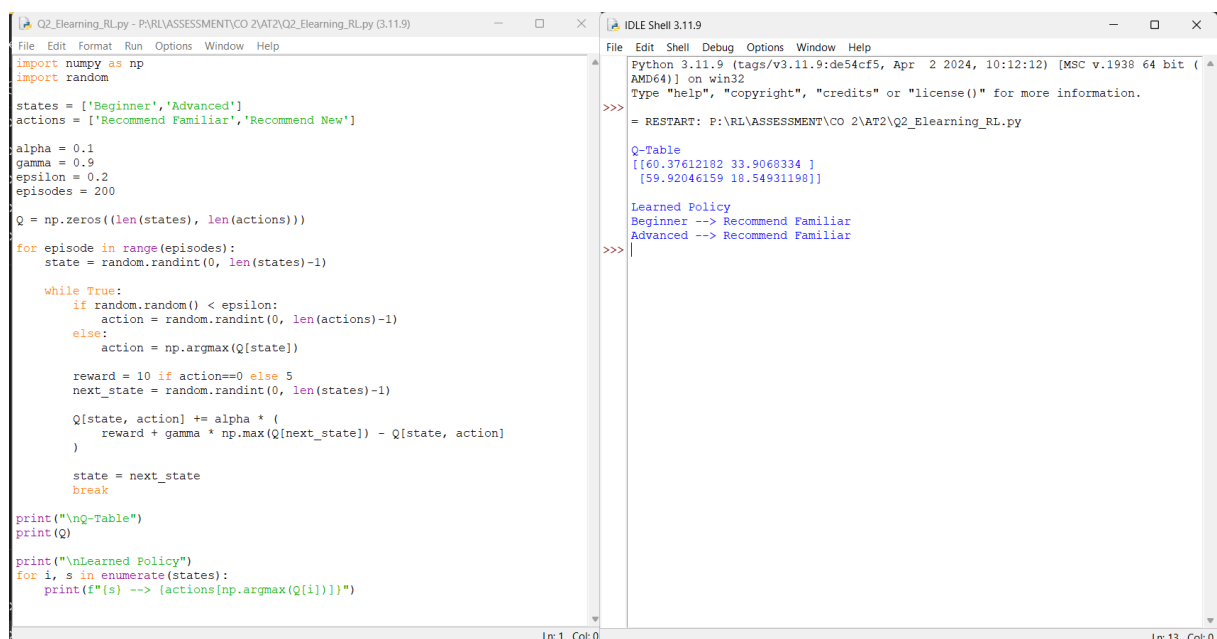
States: Beginner, Advanced

Actions: Recommend Familiar Content, Recommend New Content

Rewards: +10 for successful learning, -5 for poor engagement

Policy: Select the recommendation with the highest reward.

Code & Output



```
Q2_Elearning_RL.py - PAR\ASSESSMENT\CO 2\AT2\Q2_Elearning_RL.py (3.11.9)
File Edit Format Run Options Window Help
import numpy as np
import random

states = ['Beginner', 'Advanced']
actions = ['Recommend Familiar', 'Recommend New']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)

    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==0 else 5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        break

print("\nQ-Table")
print(Q)

print("\nLearned Policy")
for i, s in enumerate(states):
    print(f"{s} --> {actions[np.argmax(Q[i])]}")

IDLE Shell 3.11.9
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: F:\RL\ASSESSMENT\CO 2\AT2\Q2_Elearning_RL.py

Q-Table
[[60.37612182 33.9068334 ]
 [59.92046159 18.54931198]]

Learned Policy
Beginner --> Recommend Familiar
Advanced --> Recommend Familiar
>>>
```

Code & Output

1. Set exploration rate.
2. Generate random values.
3. Explore with probability ϵ .
4. Otherwise exploit.
5. Repeat multiple times.
6. Display decisions.

Advantages

- Personalized learning.
- Improves engagement.
- Balances old and new topics.
- Adapts to students.

Applications

- Coursera
- Udemy
- Byju's
- Khan Academy

Conclusion

RL balances exploration and exploitation to recommend effective learning content.

Q3. Design and develop a Reinforcement Learning framework for a robotic vacuum cleaner that uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time. (5 Marks)

Aim

To design an RL framework for an autonomous robotic vacuum cleaner.

Theory

RL enables a robot vacuum to learn efficient cleaning paths while avoiding obstacles. The value function estimates future rewards, helping the robot improve its cleaning strategy over time.

MDP Components

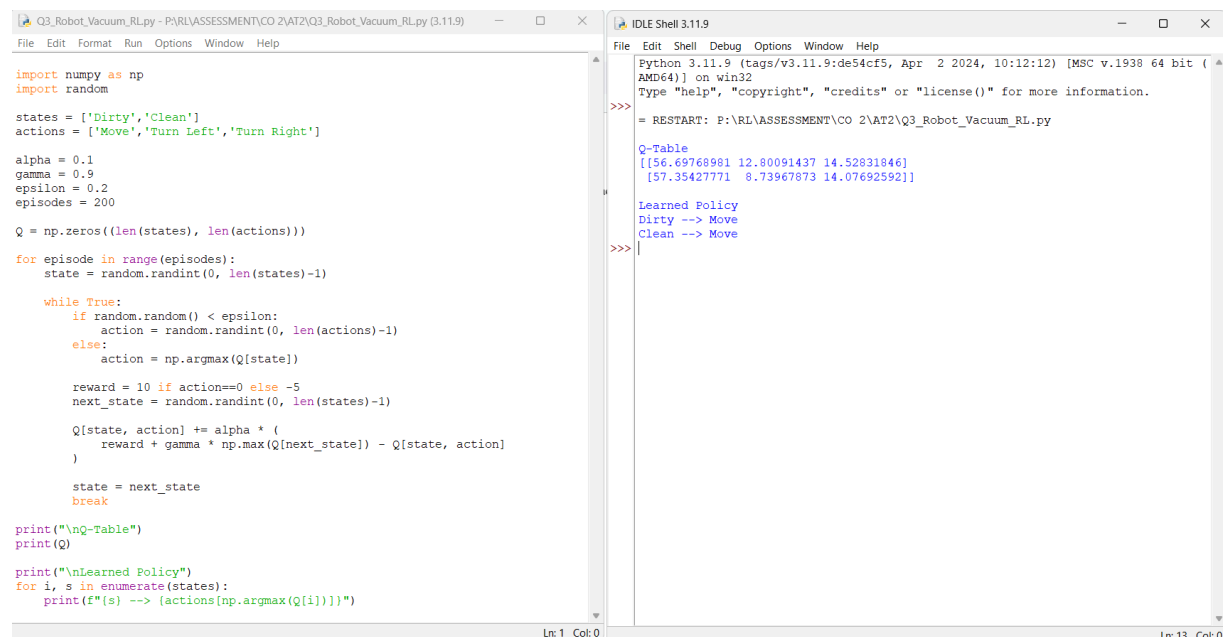
States: Clean Area, Dirty Area

Actions: Move Forward, Turn Left, Turn Right

Rewards: +10 for cleaning, -10 for hitting an obstacle

Policy: Select the action with the highest value.

Code & Output



```
import numpy as np
import random

states = ['Dirty', 'Clean']
actions = ['Move', 'Turn Left', 'Turn Right']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)

    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==0 else -5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        break

print("\nQ-Table")
print(Q)

print("\nLearned Policy")
for i, s in enumerate(states):
    print(f"{s} --> {actions[np.argmax(Q[i])]}")
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q3_Robot_Vacuum_RL.py

Q-Table
[[56.69768981 12.80091437 14.52831846]
 [57.35427771  8.73967873 14.07692592]]

Learned Policy
Dirty --> Move
Clean --> Move

>>>
```

Code & Output

1. Initialize the value function.
2. Update values repeatedly.
3. Estimate future rewards.
4. Improve the policy.

5. Learn better actions.
6. Display updated values.

Advantages

- Cleans efficiently.
- Avoids obstacles.
- Learns optimal paths.
- Saves battery power.

Applications

- Robot vacuum cleaners
- Home automation
- Service robots
- Cleaning industries

Conclusion

RL enables robotic vacuum cleaners to improve cleaning performance through continuous learning.

Q4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance. (5 Marks)

Aim

To study reward function design in cryptocurrency trading using Reinforcement Learning.

Theory

The reward function guides the RL agent in making trading decisions. A poorly designed reward may encourage risky trading, frequent unnecessary transactions, or financial losses. A well-designed reward balances profit and risk.

MDP Components

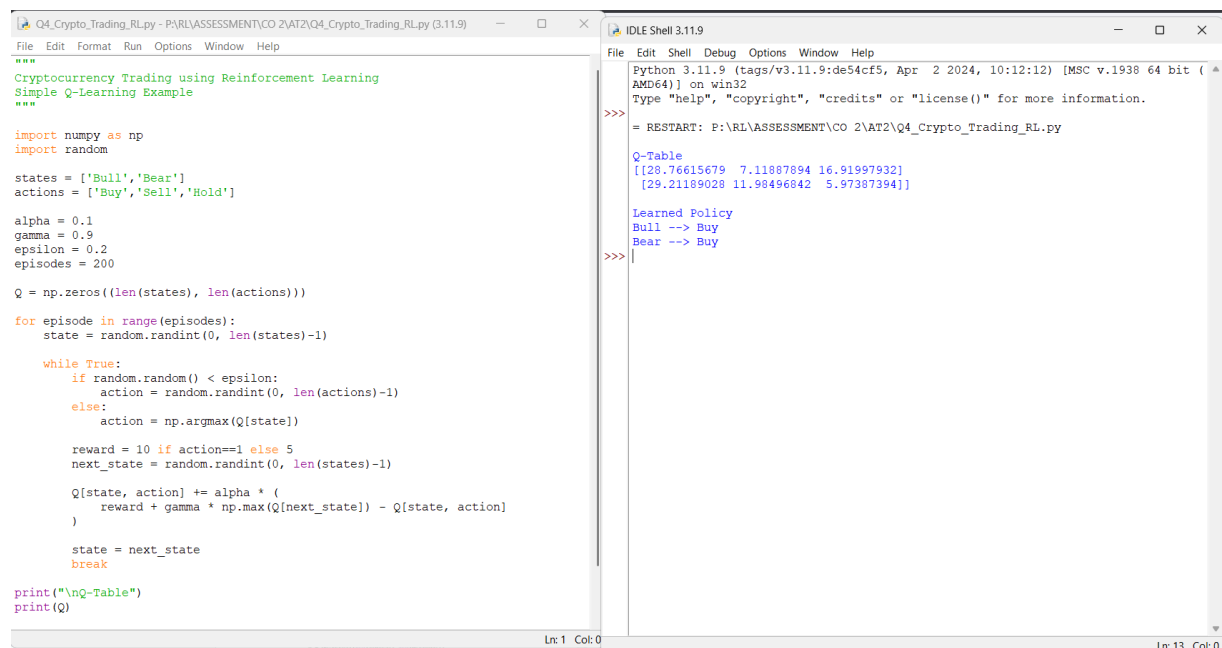
States: Price Rising, Price Falling

Actions: Buy, Sell, Hold

Rewards: +10 for profitable trades, -10 for losses

Policy: Select the trading action with the highest expected reward.

Code & Output



```
Q4_Crypto_Trading_RL.py - P:\RL\ASSESSMENT\CO 2\AT2\Q4_Crypto_Trading_RL.py (3.11.9)
File Edit Format Run Options Window Help
"""
Cryptocurrency Trading using Reinforcement Learning
Simple Q-Learning Example
"""

import numpy as np
import random

states = ['Bull', 'Bear']
actions = ['Buy', 'Sell', 'Hold']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)

    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==1 else 5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        break

print("\nQ-Table")
print(Q)
```

```
IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q4_Crypto_Trading_RL.py
Q-Table
[[28.76615679  7.11887894 16.91997932]
 [29.21189028 11.98496842  5.97387394]]

Learned Policy
Bull --> Buy
Bear --> Buy
>>>
```

Code & Output

1. Define market states.
2. Define trading actions.
3. Initialize rewards.
4. Train the RL agent.
5. Update rewards.
6. Learn the best trading policy.

Advantages

- Improves trading decisions.
- Learns market trends.
- Reduces manual effort.
- Maximizes long-term profit.

Applications

- Cryptocurrency trading
- Stock markets
- Investment platforms
- Automated trading systems

Conclusion

A properly designed reward function helps the RL agent make profitable and balanced trading decisions.

Q5. Develop a smart grid system that uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining the state space, action space, and reward mechanism. (5 Marks)

Aim

To develop an RL model for smart electricity distribution.

Theory

RL helps distribute electricity efficiently by monitoring demand and supply. The agent learns to allocate power while reducing energy waste.

MDP Components

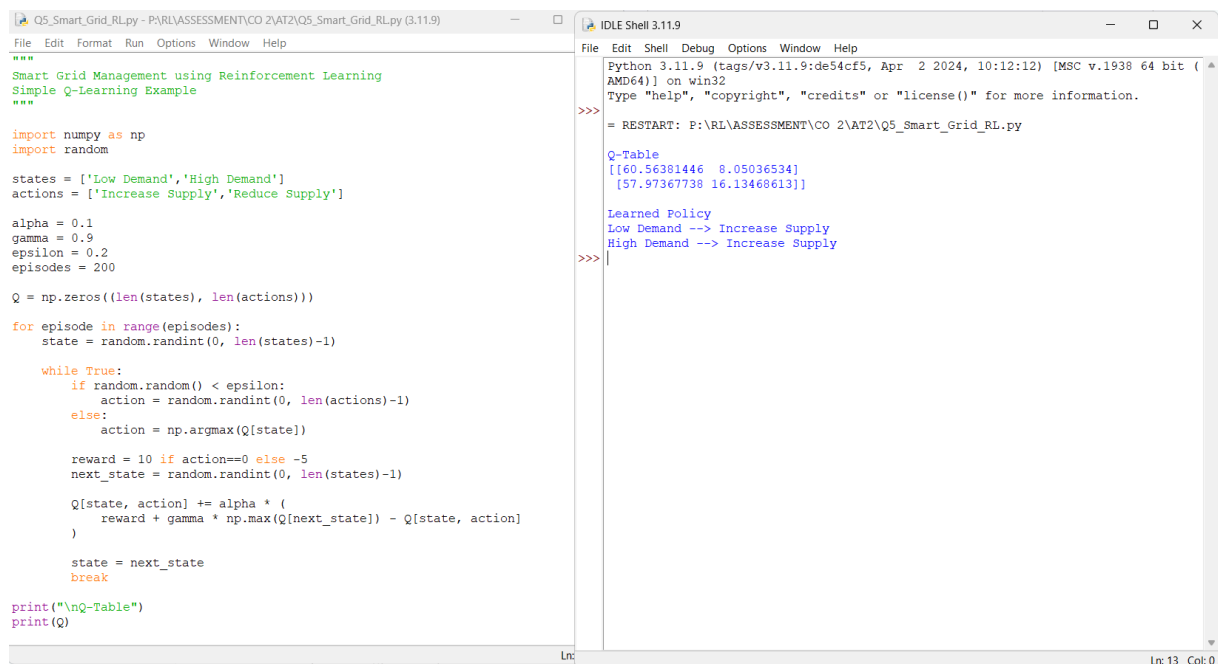
States: Low Demand, High Demand

Actions: Increase Supply, Reduce Supply

Rewards: +10 for balanced distribution, -5 for power loss

Policy: Choose the action with the highest reward.

Code & Output



The image shows a screenshot of a Python script and its output in an IDE. The script is titled "Q5_Smart_Grid_RL.py" and is located at "P:\RL\ASSESSMENT\CO 2\AT2\Q5_Smart_Grid_RL.py (3.11.9)". The script defines a Reinforcement Learning model for smart electricity distribution. It includes the following code:

```
"""
Smart Grid Management using Reinforcement Learning
Simple Q-Learning Example
"""

import numpy as np
import random

states = ['Low Demand', 'High Demand']
actions = ['Increase Supply', 'Reduce Supply']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)

    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==0 else -5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        break

print("\nQ-Table")
print(Q)
```

The output of the script is displayed in the IDE's output window. It shows the Q-Table and the Learned Policy. The Q-Table is a 2x2 matrix with the following values:

State	Low Demand	High Demand
Low Demand	60.56381446	8.05036534
High Demand	57.97367738	16.13468613

The Learned Policy is:

```
Low Demand --> Increase Supply
High Demand --> Increase Supply
```

Code Explanation

1. Define demand states.
2. Define distribution actions.
3. Initialize rewards.
4. Train the agent.
5. Update Q-values.
6. Select the best action.

Advantages

- Reduces power loss.
- Improves energy efficiency.
- Supports smart grids.
- Adapts to changing demand.

Applications

- Smart cities
- Power stations
- Renewable energy systems
- Electricity distribution

Conclusion

RL improves electricity management by learning efficient power distribution strategies.

Q6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes. (5 Marks)

Aim

To apply RL concepts for intelligent medication dosage adjustment.

Theory

RL learns the best medication dosage by monitoring a patient's health condition. Continuous learning helps improve treatment effectiveness while reducing side effects.

MDP Components

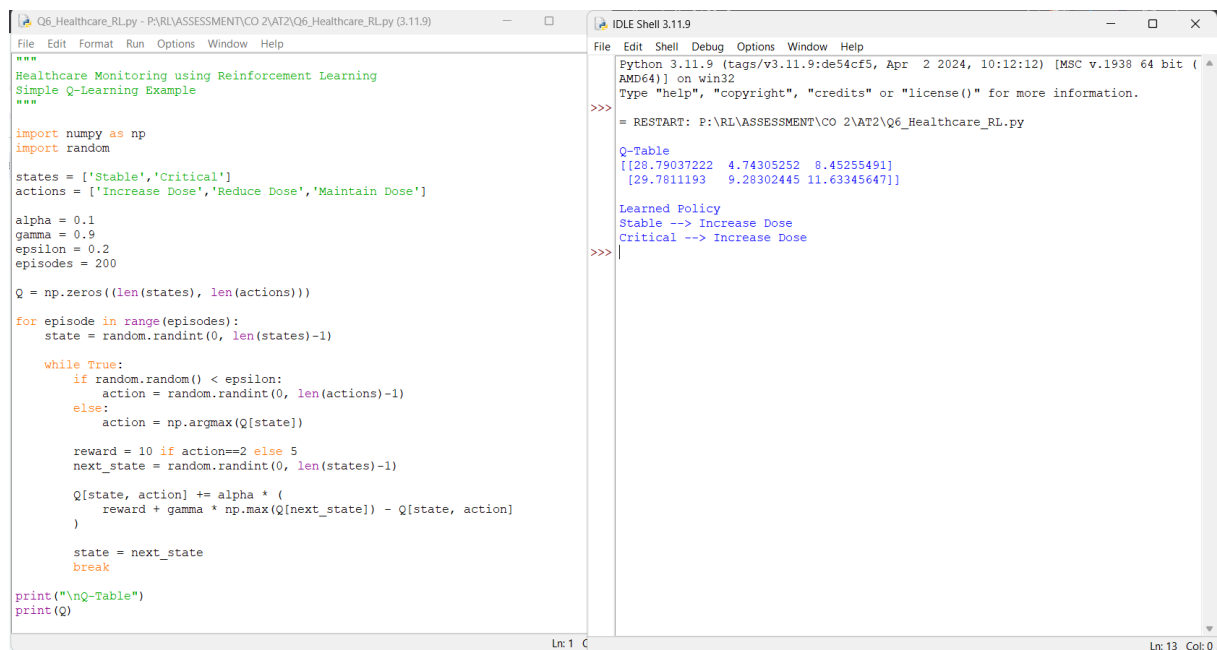
States: Stable Patient, Critical Patient

Actions: Increase Dose, Reduce Dose, Maintain Dose

Rewards: +10 for improved health, -10 for poor outcome

Policy: Select the dosage with the highest expected reward.

Code & Output



```
Q6_Healthcare_RL.py - P:\RL\ASSESSMENT\CO 2\AT2\Q6_Healthcare_RL.py (3.11.9)
File Edit Format Run Options Window Help
"""
Healthcare Monitoring using Reinforcement Learning
Simple Q-Learning Example
"""
import numpy as np
import random

states = ['Stable', 'Critical']
actions = ['Increase Dose', 'Reduce Dose', 'Maintain Dose']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)
    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==2 else 5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        break

print("\nQ-Table")
print(Q)
```

```
IDLE Shell 3.11.9
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q6_Healthcare_RL.py

Q-Table
[[28.79037222  4.74305252  8.45255491]
 [29.7811193   9.28302445 11.63345647]]

Learned Policy
Stable --> Increase Dose
Critical --> Increase Dose
>>>
```

Code Explanation

1. Define patient states.
2. Define dosage actions.
3. Initialize rewards.
4. Train the RL model.
5. Update values.
6. Learn the optimal dosage policy.

Advantages

- Personalized treatment.
- Improves patient care.
- Reduces medication errors.
- Learns continuously.

Applications

- Smart hospitals
- Healthcare monitoring
- Clinical decision systems
- Personalized medicine

Conclusion

RL improves healthcare by learning effective medication strategies based on patient conditions.

Q7. Design a drone delivery system that uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization. (5 Marks)

Aim

To design an RL-based drone navigation system.

Theory

RL helps drones discover efficient delivery routes. Exploration finds new paths, while exploitation selects the best-known routes for faster deliveries.

MDP Components

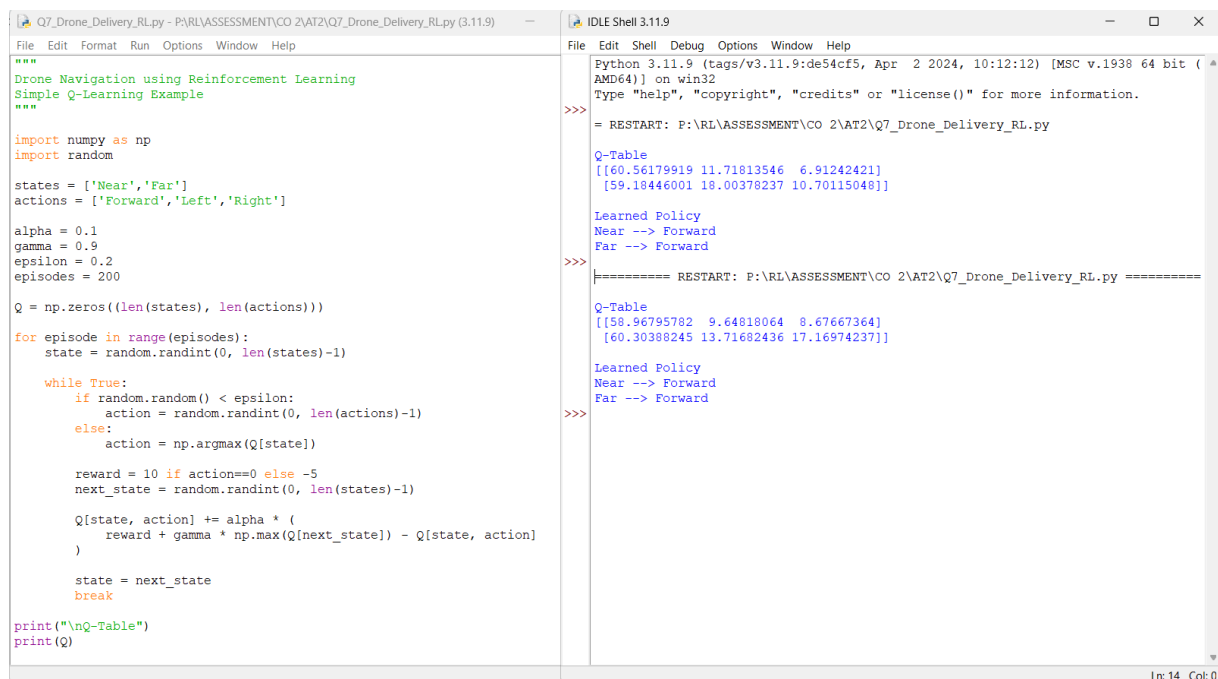
States: Destination Near, Destination Far

Actions: Move Forward, Turn Left, Turn Right

Rewards: +10 for reaching the destination, -5 for collisions

Policy: Choose the route with the highest reward.

Code & Output



The image shows a screenshot of a code editor with two windows. The left window displays a Python script for a drone navigation system using Reinforcement Learning. The script defines states ('Near', 'Far'), actions ('Forward', 'Left', 'Right'), and a Q-table. It implements a simple Q-Learning algorithm with exploration (epsilon-greedy) and exploitation. The right window shows the output of the script, which includes the initial Q-table, the learned policy, and the final Q-table after training.

```
Q7_Drone_Delivery_RL.py - P:\RL\ASSESSMENT\CO 2\AT2\Q7_Drone_Delivery_RL.py (3.11.9)
File Edit Format Run Options Window Help
"""
Drone Navigation using Reinforcement Learning
Simple Q-Learning Example
"""

import numpy as np
import random

states = ['Near', 'Far']
actions = ['Forward', 'Left', 'Right']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)

    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==0 else -5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
    break

print("\nQ-Table")
print(Q)
```

```
IDLE Shell 3.11.9
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q7_Drone_Delivery_RL.py

Q-Table
[[60.56179919 11.71813546 6.91242421]
 [59.18446001 18.00378237 10.70115048]]

Learned Policy
Near --> Forward
Far --> Forward

>>> ===== RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q7_Drone_Delivery_RL.py =====

Q-Table
[[58.96795782 9.64818064 8.67667364]
 [60.30388245 13.71682436 17.16974237]]

Learned Policy
Near --> Forward
Far --> Forward

>>>
```

Ln: 14 Col: 0

Code Explanation

1. Define navigation states.
2. Define movement actions.
3. Explore new routes.
4. Exploit the best routes.
5. Update rewards.
6. Learn the optimal path.

Advantages

- Faster deliveries.
- Efficient navigation.
- Avoids obstacles.
- Saves battery power.

Applications

- Drone delivery
- Logistics
- Medical supply transport
- Food delivery

Conclusion

RL improves drone navigation by balancing exploration and exploitation for efficient route planning.

Q8. Design and develop a Reinforcement Learning framework for a manufacturing robot that uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency. (5 Marks)

Aim

To optimize manufacturing operations using Reinforcement Learning.

Theory

RL enables manufacturing robots to learn efficient assembly sequences. The policy selects the best action, while the value function estimates future rewards to improve efficiency.

MDP Components

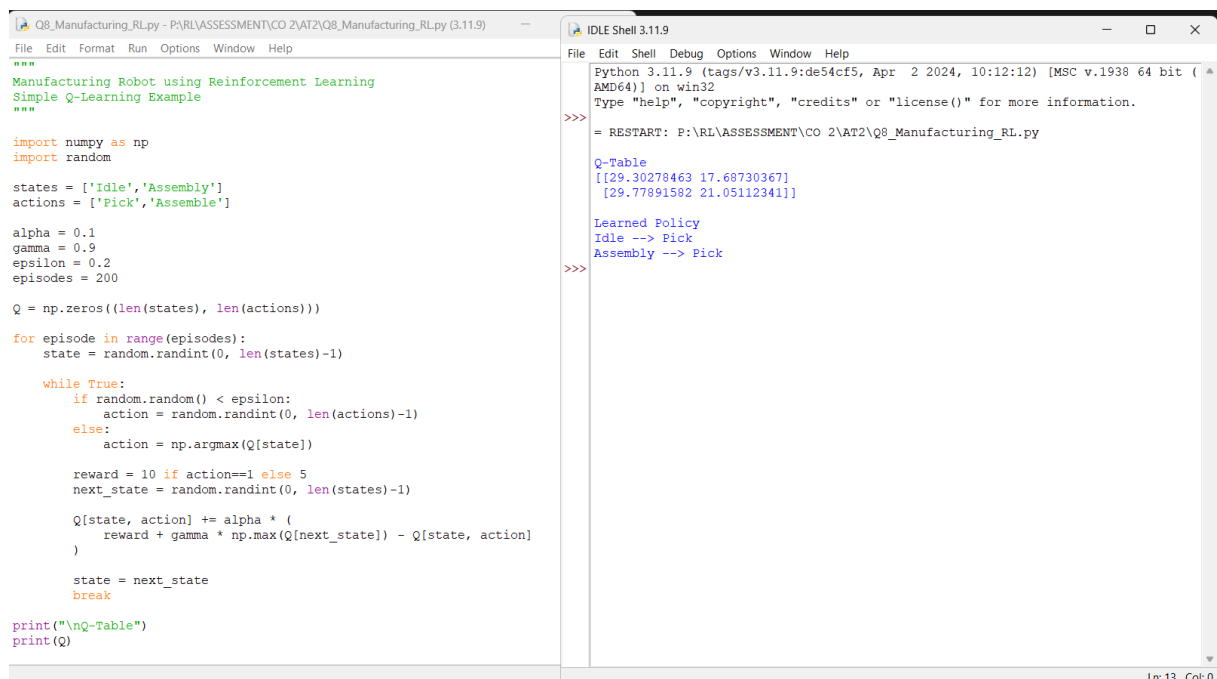
States: Assembly Started, Assembly Completed

Actions: Pick Part, Assemble Part

Rewards: +10 for successful assembly, -5 for incorrect operation

Policy: Select the action with the highest value.

Code & Output



```
Q8_Manufacturing_RL.py - P:\RL\ASSESSMENT\CO 2\AT2\Q8_Manufacturing_RL.py (3.11.9)
File Edit Format Run Options Window Help
"""
Manufacturing Robot using Reinforcement Learning
Simple Q-Learning Example
"""

import numpy as np
import random

states = ['Idle', 'Assembly']
actions = ['Pick', 'Assemble']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)

    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==1 else 5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        break

print("\nQ-Table")
print(Q)
```

```
IDLE Shell 3.11.9
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q8_Manufacturing_RL.py
Q-Table
[[29.30278463 17.68730367]
 [29.77891582 21.05112341]]
Learned Policy
Idle --> Pick
Assembly --> Pick
>>>
```

Ln: 13 Col: 0

Code Explanation

1. Define assembly states.
2. Define robot actions.
3. Initialize value estimates.
4. Update values.
5. Improve policy.
6. Learn efficient assembly.

Advantages

- Improves production speed.
- Reduces errors.
- Learns automatically.
- Increases productivity.

Applications

- Automobile manufacturing
- Electronics assembly
- Industrial robots
- Smart factories

Conclusion

RL helps manufacturing robots improve efficiency through better policies and value functions.

Q9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance. (5 Marks)

Aim

To study reward function design in online advertising.

Theory

RL recommends advertisements based on user interactions. Poor reward design may favor clicks without considering long-term engagement, reducing overall performance.

MDP Components

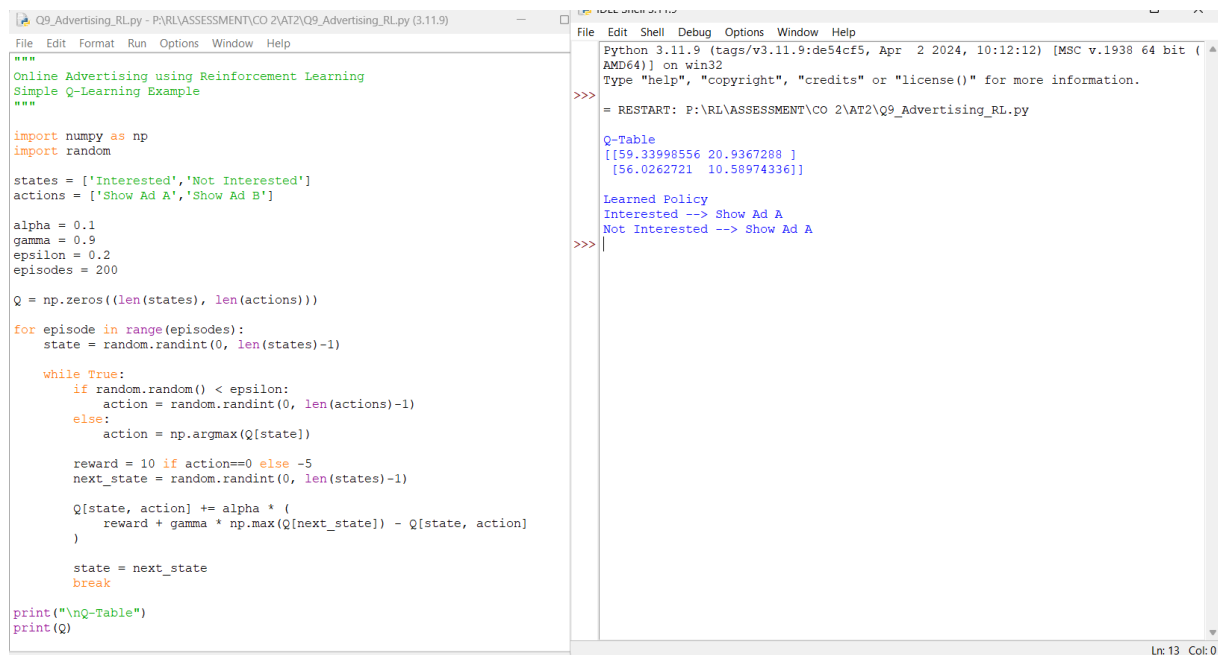
States: User Interested, User Not Interested

Actions: Show Ad A, Show Ad B

Rewards: +10 for click, -5 for no click

Policy: Select the advertisement with the highest expected reward.

Code & Output



```
Q9_Advertising_RL.py - P:\RL\ASSESSMENT\CO 2\AT2\Q9_Advertising_RL.py (3.11.9)
File Edit Format Run Options Window Help
"""
Online Advertising using Reinforcement Learning
Simple Q-Learning Example
"""
import numpy as np
import random

states = ['Interested', 'Not Interested']
actions = ['Show Ad A', 'Show Ad B']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)

    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==0 else -5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        break

print("\nQ-Table")
print(Q)
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q9_Advertising_RL.py
Q-Table
[[59.33998556 20.9367288 ]
 [56.0262721  10.58974336]]
Learned Policy
Interested --> Show Ad A
Not Interested --> Show Ad A
>>>
```

Ln: 13 Col: 0

Code Explanation

1. Define user states.
2. Define advertisement actions.
3. Initialize rewards.
4. Train the RL agent.
5. Update Q-values.
6. Learn the best advertisement policy.

Advantages

- Increases click-through rate.
- Improves user engagement.
- Learns user preferences.
- Optimizes advertising revenue.

Applications

- Google Ads
- Facebook Ads
- YouTube Ads
- E-commerce advertising

Conclusion

A well-designed reward function enables RL to improve advertisement performance and user engagement.

Q10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function. (5 Marks)

Aim

To develop an RL framework for smart irrigation.

Theory

RL controls irrigation by monitoring soil moisture and weather conditions. The agent learns when to irrigate to reduce water consumption while maintaining healthy crops.

MDP Components

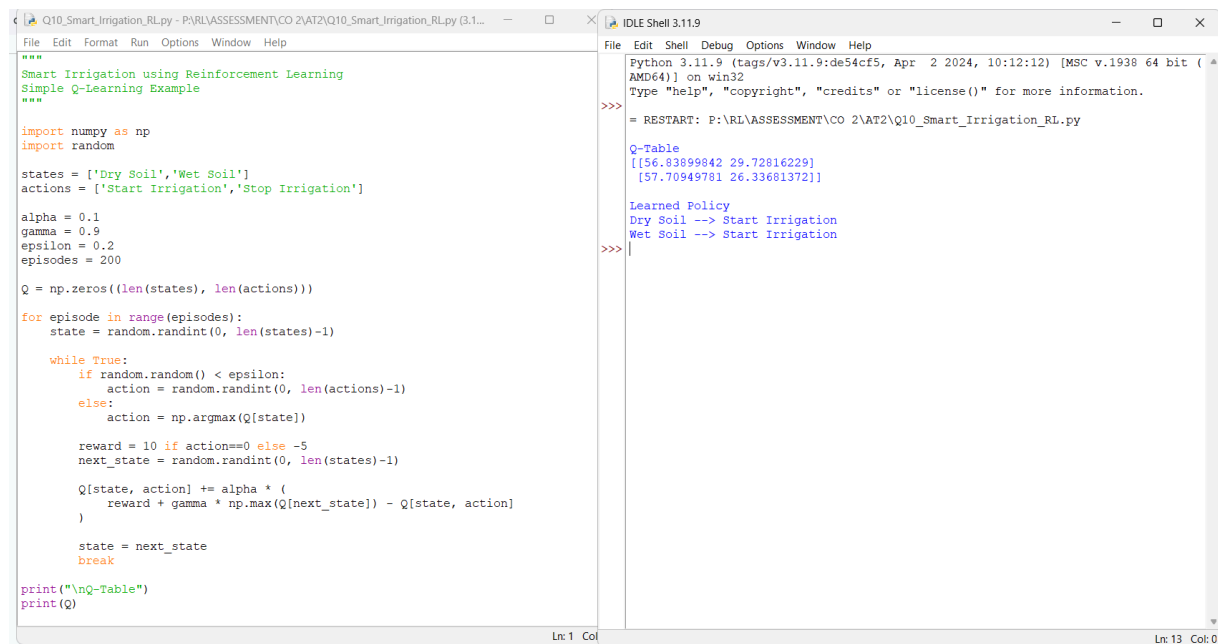
States: Dry Soil, Wet Soil

Actions: Start Irrigation, Stop Irrigation

Rewards: +10 for efficient watering, -5 for water wastage

Policy: Choose the irrigation action with the highest reward.

Code & Output



The screenshot shows a Python script in a file editor and its execution output in a shell window. The script implements a simple Q-Learning algorithm for a smart irrigation system. It defines two states: 'Dry Soil' and 'Wet Soil', and two actions: 'Start Irrigation' and 'Stop Irrigation'. The reward function gives +10 for efficient watering and -5 for water wastage. The Q-table is updated using the Bellman optimality equation. The output shows the learned policy for both states.

```
Q10_Smart_Irrigation_RL.py - PyRL\ASSESSMENT\CO 2\AT2\Q10_Smart_Irrigation_RL.py (3.11.9)
File Edit Format Run Options Window Help
"""
Smart Irrigation using Reinforcement Learning
Simple Q-Learning Example
"""

import numpy as np
import random

states = ['Dry Soil', 'Wet Soil']
actions = ['Start Irrigation', 'Stop Irrigation']

alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 200

Q = np.zeros((len(states), len(actions)))

for episode in range(episodes):
    state = random.randint(0, len(states)-1)
    while True:
        if random.random() < epsilon:
            action = random.randint(0, len(actions)-1)
        else:
            action = np.argmax(Q[state])

        reward = 10 if action==0 else -5
        next_state = random.randint(0, len(states)-1)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        break

print("\nQ-Table")
print(Q)

Ln: 1 Col: 1

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: P:\RL\ASSESSMENT\CO 2\AT2\Q10_Smart_Irrigation_RL.py

Q-Table
[[56.83899842 29.72816229]
 [57.70949781 26.33681372]]

Learned Policy
Dry Soil --> Start Irrigation
Wet Soil --> Start Irrigation

>>>

Ln: 13 Col: 0
```

Code Explanation

1. Define soil states.
2. Define irrigation actions.
3. Initialize rewards.
4. Train the RL model.
5. Update Q-values.
6. Learn the optimal irrigation policy.

Advantages

- Saves water.
- Improves crop growth.
- Reduces irrigation cost.
- Supports sustainable farming.

Applications

- Smart agriculture
- Precision farming
- Greenhouses
- Automated irrigation systems

Conclusion

Reinforcement Learning enables efficient irrigation by learning optimal watering decisions based on environmental conditions.